
Modül 23 — Spring Cloud

Ders Notları

Spring Boot Edu



2026-09-25

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Bu Modülün Sistemi	2
2.2 Spring Cloud mu, Kubernetes Native mi? Bir Karar Rehberi	3
3. Adım Adım Örnekler	4
3.1 Gateway	4
3.2 HTTP Interface mi, OpenFeign mi?	5
3.3 Spring Cloud LoadBalancer	6
3.4 Circuit Breaker ve Fallback	7
3.5 Config Server	9
3.6 Compose ile Tüm Sistem	10
3.7 Spring Cloud Kubernetes	11
4. Sık Yapılan Hatalar ve En İyi Pratikler	13
5. Özet	13
6. İleri Okuma	14

1. Öğrenme Hedefleri

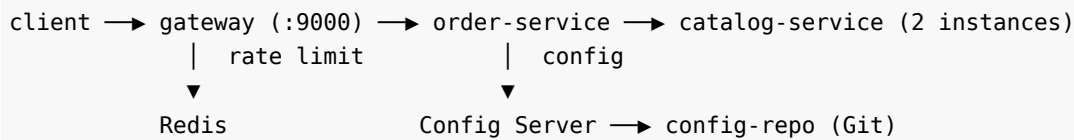
Bu modülün sonunda şunları yapabileceksiniz:

- Servislerin önüne bir API gateway koymak: Spring Cloud Gateway ile route'lar, filtreler ve rate limiting
- Tüm servislerin yapılandırmasını Git backend'li bir Config Server'dan sunmak ve onu yeniden başlatmadan yenilemek
- Başka bir servisi bir HTTP interface veya OpenFeign ile çağırmak ve ikisini karşılaştırmak
- Çağrılarını Spring Cloud LoadBalancer ile birden çok örneğe dağıtmak
- Çağırılanları bir circuit breaker (Resilience4j) ve bir fallback ile korumak
- Spring Cloud Kubernetes ile yapılandırma ve keşif için Kubernetes'in kendisini kullanmak
- Bu problemlerden hangisini Spring Cloud'un, hangisini platformun çözmesi gerektiğine karar vermek

Ön koşullar: Modül 04 (HTTP istemcileri, dayanıklılık), Modül 15 (Actuator), Modül 22 (Kubernetes) · **Tahmini süre:** 6 saat · **Docker gerekir; bölüm 3.7 için kind**

2. Kavramlar

2.1 Bu Modülün Sistemi



Klasör	Servis	Konular
services/gateway	API gateway	route'lar, filtreler, rate limiting (Redis)
services/config-server	Config Server	Git backend, profiller
lesson	sipariş servisi	HTTP interface ile Feign, LoadBalancer, circuit breaker, refresh, Spring Cloud Kubernetes
services/catalog-service	katalog	çağrılan servis; hataları simüle edebilir

2.2 Spring Cloud mu, Kubernetes Native mi? Bir Karar Rehberi

Spring Cloud dağıtık sistemlerin tipik problemlerini uygulamanın içinde çözer. Kubernetes (ve Istio ya da Linkerd gibi service mesh'ler) aynı problemlerin çoğunu platformda çözer. Hiçbiri “doğru” değildir. Önemli olan her problemin bir kez, tek bir yerde çözülmesidir.

Problem	Spring Cloud (uygulamada)	Kubernetes / platform	Platformu seçin, eğer ...
Servis keşfi	DiscoveryClient + LoadBalancer	Service + DNS	Kubernetes'te çalışıyorsanız: bir Service adı yeter
Yük dengeleme	istemci tarafında, istek başına (LoadBalancer)	kube-proxy bağlantı başına; mesh ile istek başına	istemci tarafı kurallara (zone, hint) ihtiyacınız yoksa
Yapılandırma	Config Server (Git, geçmiş, şifreleme)	ConfigMap / Secret	yapılandırma ortam başına ve manifest'lerle deploy ediliyorsa
Yapılandırma yenileme	/actuator/refresh, @RefreshScope	rolling restart (generator'lar, modül 22) veya Spring Cloud Kubernetes reload	bir yeniden başlatma ucuz ve güvenliyse
Kenar / API gateway	Spring Cloud Gateway (Java filtreleri, özel mantık)	Ingress / Gateway API controller	yalnızca routing, TLS ve basit limitlere ihtiyacınız varsa
Dayanıklılık	Resilience4j (kodda fallback)	mesh retry'ları ve timeout'ları	tepki iş mantığı gerektirmiyorsa (bir fallback gerektirir)

Pratik kurallar:

- **Kubernetes dışında** (VM'ler, Docker Compose) Spring Cloud, platformun vermediğini verir.
- **Kubernetes'te** keşif ve temel yapılandırma için platformu kullanın. Spring Cloud'u uygulamanın bir şeye karar vermesi gereken yerde tutun: iş anlamı olan bir fallback, uygulama mantığı olan bir gateway filtresi, birçok servis arasında yapılandırma geçmişi.
- **Spring Cloud Kubernetes** köprüdür: Aynı Spring soyutlamaları (DiscoveryClient, property source'lar, refresh), arka uç olarak Kubernetes ile.

3. Adım Adım Örnekler

Tüm sistemi Docker Compose ile başlatın. Script önce dört servisi build eder:

```
modules/23-spring-cloud/up.sh
```

```
bookstore-cloud-gateway-1      Up (healthy)    0.0.0.0:9000->8080/tcp
bookstore-cloud-order-service-1 Up (healthy)    8080/tcp
bookstore-cloud-catalog-1-1    Up (healthy)    8080/tcp
bookstore-cloud-catalog-2-1    Up (healthy)    8080/tcp
bookstore-cloud-config-server-1 Up (healthy)    8080/tcp
bookstore-cloud-redis-1        Up (healthy)    6379/tcp
```

Yalnızca gateway bir port yayınlr. [requests.http](#) bu bölümün tüm isteklerini içerir.

3.1 Gateway

services/gateway/src/main/resources/application.yaml'nin route'ları (spring: altında):

```
cloud:
  gateway:
    server:
      webflux:
        default-filters:
          - AddResponseHeader=X-Served-Through, bookstore-gateway
        routes:
          - id: catalog
            uri: lb://catalog-service          # lb:// → Spring Cloud LoadBalancer
            ↪ picks an instance
            predicates:
              - Path=/api/books/**
            filters:
              - name: RequestRateLimiter      # token bucket in Redis, shared by all
                ↪ gateway instances
                args:
                  redis-rate-limiter.replenishRate: 5    # tokens per second
                  redis-rate-limiter.burstCapacity: 10   # at most 10 requests at once
                  key-resolver: "#{@clientIpKeyResolver}"
          - id: orders
            uri: lb://order-service
            predicates:
              - Path=/api/orders/**
              - Method=POST
```

- Bir **route**'un predicate'leri (burada path) ve bir hedefi vardır. lb://catalog-service load balancer'dan bir örnek ister.
- **Filtreler** isteği veya yanıtı değiştirir. default-filters tüm route'lara uygulanır.
- RequestRateLimiter, Redis'te bir token bucket'tır. Birden çok gateway örneği onu paylaşır.

Anahtar, kimin isteklerinin birlikte sayılacağına karar verir:

```
@Configuration(proxyBeanMethods = false)
class RateLimitConfiguration {

    @Bean
    KeyResolver clientIpKeyResolver() {
        return exchange ->
            ↪ Mono.just(Optional.ofNullable(exchange.getRequest().getHeaders().getFirst("X-
            ↪ Customer")))
                .or(() -> Opti-
                ↪ onal.ofNullable(exchange.getRequest().getRemoteAddress()).map(InetSocketAddress::g
                .orElse("unknown"));
    }
}
```

```
X-RateLimit-Remaining: 5
X-RateLimit-Burst-Capacity: 10
X-RateLimit-Replenish-Rate: 5
X-Served-Through: bookstore-gateway
```

Bucket boşaldığında gateway 429 Too Many Requests cevabı verir ve servisi çağırılmaz:

```
@Test
void tooManyRequestsAreRejected() {
    List<Integer> statuses = new ArrayList<>();
    for (int i = 0; i < 20; i++) { // burst capacity 10, 5 new tokens
        ↪ per second
        statuses.add(http.get().uri("/api/books/9780134685991").header("X-Customer",
        ↪ "rate-limit-test").exchange()
            .returnResult(String.class).getStatus().value());
    }

    assertThat(statuses).contains(200, 429); // 429 Too Many Requests
}
```

3.2 HTTP Interface mi, OpenFeign mi?

Sipariş servisi fiyatı katalogdan sorar. Aynı istemci, iki şekilde yazılmış:

```
@HttpExchange("/api/books")
public interface CatalogHttpClient {

    @GetExchange("/{isbn}")
    Book find(@PathVariable String isbn);
}
```

```
@FeignClient(name = "catalog-service", path = "/api/books")    // name = the service ID,
↳ resolved by the load balancer
public interface CatalogFeignClient {

    @GetMapping("/{isbn}")
    Book find(@PathVariable String isbn);
}
```

	HTTP interface	OpenFeign
Parçası olduğu	Spring Framework (6+)	Spring Cloud (Netflix mirası)
Anotasyonlar	@HttpExchange, @GetExchange	Spring MVC anotasyonları (@GetMapping)
Altta ki istemci	RestClient / WebClient (senkron veya reaktif)	Feign'in kendi istemcisi, bloklayan
Yük dengeleme	@LoadBalanced builder (bölüm 3.3)	yerleşik (name = servis ID'si)
Durum	yeni kod için önerilen yol	2022.0'dan beri "feature-complete": yalnızca hata düzeltmeleri; Spring ekibi HTTP service client'larını öneriyor

İkisi de aynı kitabı verir (LoadBalancedClientsTest). Yeni kod için HTTP interface'leri kullanın. Feign mevcut sistemlerde hâlâ yaygındır.

3.3 Spring Cloud LoadBalancer

HTTP interface, yük dengelemesini @LoadBalanced bir RestClient.Builder'dan alır. `http://catalog-service`'in host'u bir DNS adı değil, bir servis ID'sidir:

```

@Configuration(proxyBeanMethods = false)
class CatalogClientConfiguration {

    @Bean
    @LoadBalanced // the host name is a service ID,
    ↪ not a DNS name
    RestClient.Builder loadBalancedRestClientBuilder() {
        return RestClient.builder();
    }

    @Bean
    CatalogHttpClient catalogHttpClient(RestClient.Builder
    ↪ loadBalancedRestClientBuilder) {
        RestClient restClient =
    ↪ loadBalancedRestClientBuilder.baseUrl("http://catalog-service").build();
        return HttpServiceProxyFac-
        ↪ tory.builderFor(RestClientAdapter.create(restClient)).build()
        .createClient(CatalogHttpClient.class);
    }
}

```

Kubernetes dışında örnekler sabit bir listeden gelir (SimpleDiscoveryClient). Kubernetes'te onları küme sağlar (bölüm 3.7):

```

# Lesson 3.3 – outside Kubernetes: a fixed list of instances (spring-boot:run,
↪ compose.yaml)
spring:
  config:
    activate:
      on-profile: "!kubernetes"
  cloud:
    discovery:
      client:
        simple:
          instances:
            catalog-service:
              - uri: ${CATALOG_1_URL:http://localhost:8081}
              - uri: ${CATALOG_2_URL:http://localhost:8082}

```

Katalogu gateway üzerinden dört kez çağırın: servedBy, catalog-1 ile catalog-2 arasında değişir (round robin).

3.4 Circuit Breaker ve Fallback

Katalog başarısız olduğunda sipariş servisi ne beklemeli ne de başarısız olmalıdır. Siparişi PENDING olarak kabul eder ve daha sonra fiyatlandırır:

```

public OrderResponse place(OrderRequest request, String client) {
    if (request.quantity() > limits.maxQuantity()) {
        throw new IllegalArgumentException("At most " + limits.maxQuantity() + " copies
        ↪ per order");
    }
    Supplier<Book> call = "feign".equals(client)
        ? () -> feignClient.find(request.isbn())
        : () -> httpClient.find(request.isbn());
    return catalogBreaker.run(
        () -> priced(request, call.get(), client),
        error -> pending(request, client));    // fallback: failure, timeout, or
    ↪ the breaker is open
}

```

```

resilience4j:
  circuitbreaker:
    instances:
      catalog:
        sliding-window-size: 4           # look at the last 4 calls ...
        minimum-number-of-calls: 4
        failure-rate-threshold: 50      # ... and open when half of them failed
        wait-duration-in-open-state: 10s # then try again after 10 s (HALF_OPEN)
      timelimiter:
        instances:
          catalog:
            timeout-duration: 1s        # a slower answer counts as a failure

```

Durum	Davranış
CLOSED	çağrılar geçer; hatalar sayılır
OPEN	çağrılar hemen başarısız olur, fallback cevap verir, katalog çağrılmaz
HALF_OPEN	wait-duration-in-open-state sonrasında birkaç deneme çağrısı karar verir: CLOSED'a dönüş veya yeniden OPEN

```

@Test
void failuresOpenTheBreakerAndTheFallbackAnswers() {
    CatalogInstances.failing();

    for (int i = 0; i < 4; i++) {           // minimum-number-of-calls: 4
        assertThat(orders.place(new OrderRequest("9780134685991", 1), "http").status())
            .isEqualTo(OrderResponse.Status.PENDING);
    }
}

```

```

    }

    @Test
    void hat() {
        ↪ hat(breakers.circuitBreaker("catalog").getState()).isEqualTo(CircuitBreaker.State.OPEN);
        int callsBefore = CatalogInstances.CATALOG_1.getAllServeEvents().size()
            + CatalogInstances.CATALOG_2.getAllServeEvents().size();
        orders.place(new OrderRequest("9780134685991", 1), "http");
        int callsAfter = CatalogInstances.CATALOG_1.getAllServeEvents().size()
            + CatalogInstances.CATALOG_2.getAllServeEvents().size();
        assertThat(callsAfter).isEqualTo(callsBefore); // open: the catalog is not
        ↪ called at all
    }

```

Çalışan sistemle deneyin: İki katalog örneğini de hata moduna alın (bkz. requests.http), siparişler gönderin ve sipariş servisinin /actuator/circuitbreakers'ına bakın: "state":"OPEN".

3.5 Config Server

```

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

```

```

cloud:
  config:
    server:
      git:
        # the course repository itself (run from this folder); in production: the URL of
        ↪ a config repository
        uri: ${CONFIG_GIT_URI:file://${user.dir}/../../../../../}
        search-paths: modules/23-spring-cloud/config-repo
        default-label: main # the branch – only committed changes are visible
        clone-on-start: true

```

Sunucu GET /order-service/default'a config-repo/'daki order-service.yaml'nin (ve tüm servisler için application.yaml'nin) içeriğiyle cevap verir. Bir profil order-service-prod.yaml'yi ekler. ConfigServerTest JGit ile geçici bir Git deposu oluşturur ve ikisini de kontrol eder.

İstemci yapılandırmayı başlangıçta import eder:

```
config:
  import: optional:configserver:${CONFIG_SERVER_URL:http://localhost:8888} # optional:
  ↪ starts without it
```

Bir değer bir refresh'ten sonra yalnızca **refresh scope**'taki bean'ler tarafından yeniden okunur:

```
@Component
@RefreshScope
public class OrderLimits {

    private final int maxQuantity;

    OrderLimits(@Value("${bookstore.order.max-quantity:10}") int maxQuantity) {
        this.maxQuantity = maxQuantity;
    }

    public int maxQuantity() {
        return maxQuantity;
    }
}
```

```
# change bookstore.order.max-quantity in config-repo/order-service.yaml (and commit,
↪ with the Git backend)
docker exec bookstore-cloud-order-service-1 wget -q0- --post-data='
↪ http://localhost:8080/actuator/refresh
["bookstore.order.max-quantity"]
```

i Not

Git backend ile sunucu yalnızca **commit edilmiş** değişiklikleri görür: Yapılandırma, kod gibi bir geçmiş, review'lar ve geri almalar kazanır. Compose container'ları bunun yerine native profilini (bağlanmış bir klasör) kullanır, böylece demo container içinde Git'e ihtiyaç duymaz.

! Uyarı

macOS'ta /var bir sembolik linktir ve Config Server sembolik link içeren depo yollarını reddeder. Bu yüzden test, geçici deposu için toRealPath() kullanır.

3.6 Compose ile Tüm Sistem

compose.yaml tüm servisleri tek bir Dockerfile ile (modül 20'deki) build eder ve onları health check'lerle doğru sırada başlatır: önce Redis ve Config Server, en son gateway. Ser-

visler birbirlerini compose adlarıyla bulur (<http://catalog-1:8080>).

Sistemi docker compose -f modules/23-spring-cloud/compose.yaml down ile durdurun.

3.7 Spring Cloud Kubernetes

Kubernetes'te sipariş servisinin ne Config Server'a ne de sabit örnek listesine ihtiyacı vardır. Spring Cloud Kubernetes ikisini de Kubernetes API'sinden okur:

```
# Lesson 3.7 – on Kubernetes: the ConfigMap "order-service" is a property source, the
↪ pods of the Service
# "catalog-service" are the instances, and a changed ConfigMap refreshes the application
↪ without a restart
spring:
  config:
    activate:
      on-profile: kubernetes
    import: "kubernetes:"
  cloud:
    config:
      enabled: false # no Config Server on Kubernetes
    kubernetes:
      reload:
        enabled: true
        mode: event # watch the ConfigMap (needs the "watch" permission)
      loadbalancer:
        mode: POD # the pods behind the Service are the instances:
↪ Spring Cloud balances
```

ConfigMap uygulamanın adını taşır:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: order-service # = spring.application.name: read as a
↪ property source
data:
  application.yaml: |
    bookstore:
      order:
        max-quantity: 4
```

Pod Kubernetes API'sini okur, bu yüzden izinlere ihtiyaç duyar:

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: order-service
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: spring-cloud-kubernetes
rules:
  - apiGroups: [""]
    resources: ["configmaps", "services", "endpoints", "pods"]
    verbs: ["get", "list", "watch"]           # watch: for the reload of changed
    ↪ ConfigMaps
  - apiGroups: ["discovery.k8s.io"]
    resources: ["endpointslices"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: order-service-reads-the-api
subjects:
  - kind: ServiceAccount
    name: order-service
roleRef:
  kind: Role
  name: spring-cloud-kubernetes
  apiGroup: rbac.authorization.k8s.io

```

```

scripts/kind-up.sh
modules/23-spring-cloud/up.sh           # builds the images (the compose system can be
    ↪ stopped afterwards)
modules/23-spring-cloud/deploy-k8s.sh
kubectl -n bookstore-cloud port-forward service/order-service 18090:8080

```

kind’da gözlemlediklerimiz:

- 5 kopyalık bir sipariş: At most 4 copies per order (ConfigMap’ten gelen değer).
- Dört sipariş: pricedBy iki katalog pod’u arasında değişir (mode: POD).
- max-quantity: 6 ile `kubectl -n bookstore-cloud patch configmap order-service ...`: Birkaç saniye sonra 5 kopya, yeniden başlatma olmadan kabul edilir (reload.mode: event).

★ **İpucu**

`loadbalancer.mode: SERVICE` ile sipariş servisi Kubernetes Service'ini çağırır ve kube-proxy **bağlantı başına** dengeler. HTTP bağlantıları açık tutulduğu için tüm çağrılar aynı pod'a gitti. POD modu, Spring Cloud LoadBalancer'ın istek başına seçmesini sağlar.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! **Dikkat**

Bir fallback “bir şey” değil, iş açısından doğru olmalıdır. Daha sonra fiyatlandırılan PENDING bir sipariş dürüştür. Uydurulmuş bir fiyat değildir.

- **Yapın:** Her problemi tek bir yerde çözün: Aynı çağrılar için keşfi Kubernetes veya Spring Cloud ile, ikisiyle aynı anda değil.
- **Yapmayın:** Gateway'e iş mantığı koymayın. O yönlendirir, kimlik doğrular ve sınırlar. Siparişler sipariş servisinin işidir.
- **Yapın:** Her uzak çağrıya bir timeout verin (burada TimeLimiter, 1 s). Timeout'suz bir circuit breaker yavaş servislere karşı korumaz.
- **Yapmayın:** /actuator/refresh'i herkese açmayın. Çalışan sistemi değiştirir.
- **Yapın:** Sırları Config Server'ın düz dosyalarının dışında tutun (şifrelemesini, Vault'u veya Kubernetes Secret'lerini kullanın).
- **Yapmayın:** Yeni koda Feign ile başlamayın. HTTP interface'leri Spring Framework standardıdır.

5. Özet

- Spring Cloud Gateway predicate'lere göre yönlendirir, istekleri filtrelerle değiştirir ve hızı Redis'teki bir token bucket ile sınırlar.
- Config Server yapılandırmayı Git'ten sunar. @RefreshScope bean'leri /actuator/refresh sonrasında yeni değerleri alır.
- HTTP interface'leri ve OpenFeign istemcileri arayüz olarak tarif eder. @LoadBalanced ve lb:// servis ID'lerini örneklere çözer.
- Bir circuit breaker hataları sayar, açılır ve servis toparlanana kadar bir fallback'in cevap vermesini sağlar.
- Spring Cloud Kubernetes, ConfigMap'leri ve Kubernetes API'sini aynı Spring soyutlamalarıyla kullanır.
- Kubernetes'te keşif ve basit yapılandırma için platformu tercih edin, Spring Cloud'u uygulamanın karar verdiği yerde tutun.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Cloud Gateway](#) · [Spring Cloud Config](#)
- [Spring Cloud Commons — LoadBalancer](#)
- [Spring Cloud Circuit Breaker](#) · [Spring Cloud OpenFeign](#)
- [Spring Framework — HTTP Interface Clients](#)
- [Spring Cloud Kubernetes](#)